



Dynplayer

✓ 속성 4개 더 보기

dynplayer.win (Desktop recommended)

Service Overview

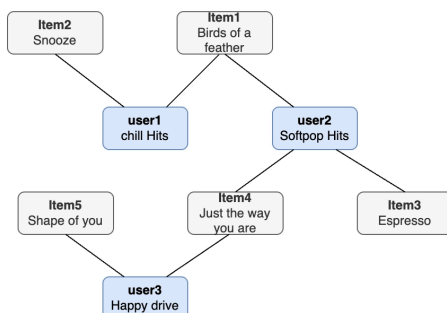
GOAL: Build a service that maximizes user engagement

- Implement a **Pinterest-style UI that encourages continuous exploration**.
To achieve this, we designed a screen that invites endless digging through reroll animations and vintage-style cover treatments.
- To understand real users' music listening patterns, we **assumed playlist data as a proxy for actual user behavior logs**.
- **Recommendation: SimGCL-based track embeddings + a Two-Tower model that reflects user sequences**
- **Keyword search: CLIP-style alignment between track embeddings and keyword embeddings + preprocessing optimized for short keyword queries**

Graph Embedding-based Music Recommendation

DATA

- Built by crawling Spotify playlist data
- **Approximately 80K tracks and 40K playlists**
- Each **playlist is treated as a single user sequence** to formulate the recommendation problem
- Constructed a user-item bipartite graph used for graph embedding training



Graph Embedding

- **Item2Vec**
 - Learns **track embeddings based on track co-occurrence within playlists (Skip-gram + negative sampling)**
 - Tracks that appear together are pulled closer, while random tracks are pushed apart
 - Performance improved significantly when contrastive learning was applied
 - However, it has the **limitation of not directly leveraging graph neighborhood structure**, which led us to explore graph embeddings

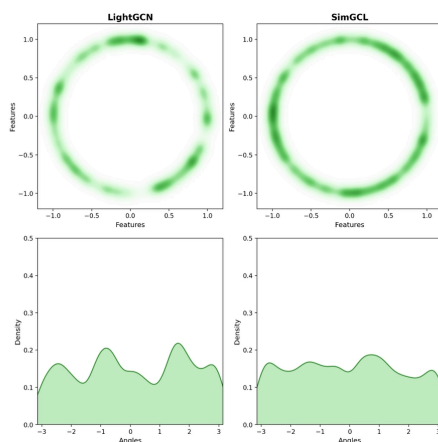
- **LightGCN**

- Learns representations on a user–item bipartite graph by repeatedly propagating (aggregating) the average of neighboring node embeddings
- **Optimizes so that user–positive items are close and negative items are far apart (BPR)**
- **Improved Recall performance**, however
- The embedding space is not uniform, and there is a stark performance gap between popular and unpopular tracks

- **SimGCL**

KDE plot after creating 2D vectors with t-SNE

- **Adds random perturbations (noise) to the LightGCN architecture** to generate embeddings from different views
- **Contrastive learning between the two views (InfoNCE)** trains the same node to be closer and different nodes to be farther apart
- **Improves uniformity** in the embedding space
 - $\mathcal{L}_{\text{uniform}} = \log \mathbb{E}_{x,y \sim p_{\text{data}}} \left[e^{-2\|f(x)-f(y)\|^2} \right]$
- As a result, forms a more **balanced representation**



KDE plot after creating 2D vectors with t-SNE

Trained by leaving out the last track in a playlist and predicting it from the remaining ones — **Leave-One-Out (LOO) evaluation**

Overall Performance Comparison

Metric	Item2vec	item2vec(CL)	LightGCN	simGCL
Recall@10	0.0486	0.1386	0.1793	0.2291
Recall@20	0.0708	0.2103	0.2523	0.3082
NDCG@10	0.0262	0.0745	0.1006	0.1367
NDCG@20	0.0318	0.0926	0.1190	0.1565

Table 1: Recall@10

Model	All	Top 10%	Bottom 10%
LightGCN	17.93	26.28	15.36
SimGCL	22.91	28.01	23.73

→ Significant improvement especially for unpopular tracks (long-tail)

Table 2: Uniformity by Popularity

Model	All	Top 10%	Bottom 10%	Gap
LightGCN	-3.71	-3.29	-3.69	0.40
SimGCL	-3.85	-3.74	-3.83	0.09

→ SimGCL produces a more uniform representation distribution for popular items, mitigating popularity bias and consequently improving recall

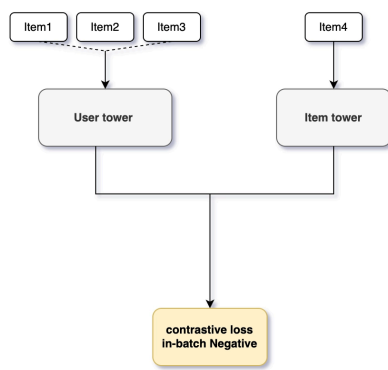
Table 3: Alignment by Popularity (between tracks in same playlists)

Model	All	Top 10% (Popular)	Bottom 10% (Unpopular)
LightGCN	0.8112	0.8457	0.4877
SimGCL	0.9209	0.9725	0.6922

Since graph models learn user embeddings statically, they cannot produce dynamic user representations based on the current session.

To address this, we introduce a Two-Tower model that generates user embeddings in real time from track embeddings within a session.

Two-Tower Model



- For playlists, context matters more than order: we use bidirectional attention
- Also, since real user sequences are likely to be shorter than playlists, we trained on cropped subsequences from playlists
- **Architecture**
 - **User Tower**
 - 2-layer Transformer Encoder
 - User **track sequence** → **user embedding**
 - **Item Tower**
 - Transforms track embeddings via an **MLP** to produce embeddings in the same shared space
- **Training**
 - **In-batch negative sampling + InfoNCE**
- **Serving**
 - Item embeddings are **pre-stored in the DB**
 - Fast recommendation via **ANN search**
- **Evaluation (LOO)**

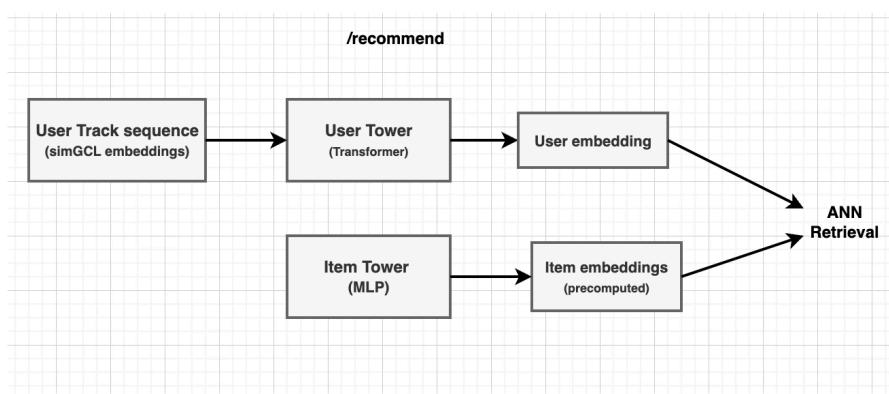
The embeddings that performed best in graph learning likewise yield the strongest performance here.

	Recall@10	Recall@20	NDCG@10	NDCG@20
Two-Tower + LightGCN	0.1604	0.2248	0.0935	0.1098
Two-Tower + SimGCL	0.1926	0.2683	0.1144	0.1335

HNSW Results (m=16, ef_construction=64)

Vector latency reduction: 86.8ms → 42.0ms

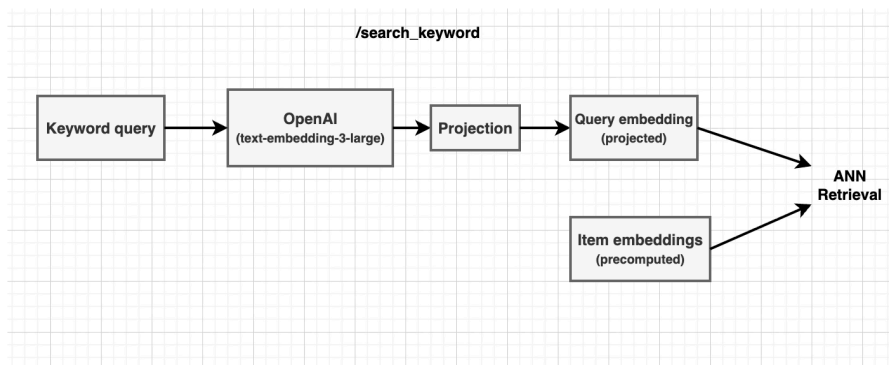
Metric	Recall
Recall@1	99.0%
Recall@5	99.0%
Recall@10	98.9%
Recall@20	98.85%



Keyword Search

GOAL: Recommend tracks that match the mood/genre the user searches for!

- **Training**
 - Playlist titles are mostly long (e.g., "Chill hip-hop/ballads for snowy winter days," "70s Hits - The Biggest Hits of the 70's")
 - But user queries will likely be **short keywords** (e.g., "ballad," "Christmas," etc.)
 - We used the GPT API to extract short keywords from playlist titles and used these for training to strengthen keyword search
70s Hits - The Biggest Hits of the 70's → ['70s', 'hits']
We then used roughly 2,300 tags that appeared 3 or more times for training.
- **Inference**
 - At inference time, we apply a projection model to track embeddings that live in the same space as playlist embeddings.
 - Projected embeddings are stored in advance to reduce search time.
 - ANN search (HNSW)
 - A ranking model (reranking) is applied to reorder initial search candidates, improving search quality



References

<https://arxiv.org/abs/1603.04259>

<https://arxiv.org/abs/2002.02126>

<https://arxiv.org/abs/2112.08679>

<https://arxiv.org/pdf/2103.00020>